Informe Técnico
Simulador del clúster del Ferrari F40 (1987)

Proyecto final de Sistemas Operativos. Software simulado: sin ECU ni sensores reales; rangos didácticos.

1. Portada e identificación del proyecto

Campo	Valor
Institución	Universidad Autónoma de Guadalajara (UAG)
Materia	Sistemas Operativos
Profesor	José de Jesús Velázquez Martínez
Equipo	Equipo 3
Integrantes	César Saúl Vázquez Preciado (5079359) Ian Josué Cota Flores (5060667)
Fecha de entrega	18 de octubre de 2026
Vehículo	Ferrari F40, 1987
Fecha de ejecución de las pruebas	2026-11-10
Versión del software	commit efc89ad
Repositorio	https://github.com/ayocesaar/cluster-f40 (privado)
Colaboración	Claude Code (Anthropic), asistente de inteligencia artificial; ver anexo

Documentos relacionados: Informe de pruebas (informe/Informe_Pruebas_Cluster_F40.pdf), matriz de trazabilidad (docs/matriz_trazabilidad.md), README.md y carpeta estudio/.

2. Objetivo, alcance y vehículo seleccionado

Objetivo: simular el clúster de instrumentos de un Ferrari F40 (1987) que recibe sus diez valores desde un archivo externo, los valida, actualiza el tablero sin reiniciar, registra eventos y errores en bitácoras separadas y aplica mecanismos de sistema operativo: hilos, comunicación productor-consumidor, escritura atómica, permisos y rotación de archivos.

- Dentro del alcance: entrada por archivo, configuración externa, validación, modelo con reglas R01–R08, interfaz Tkinter 8-bit, bitácoras, pruebas P01–P12 y métricas.
- Fuera del alcance: hardware, ECU y sensores reales; calibraciones del F40; plataformas distintas de Linux y Windows.
- La interfaz sigue la distribución del tablero del F40: velocímetro y tacómetro grandes al centro, medidores chicos a los lados y los testigos críticos arriba, entre las esferas.

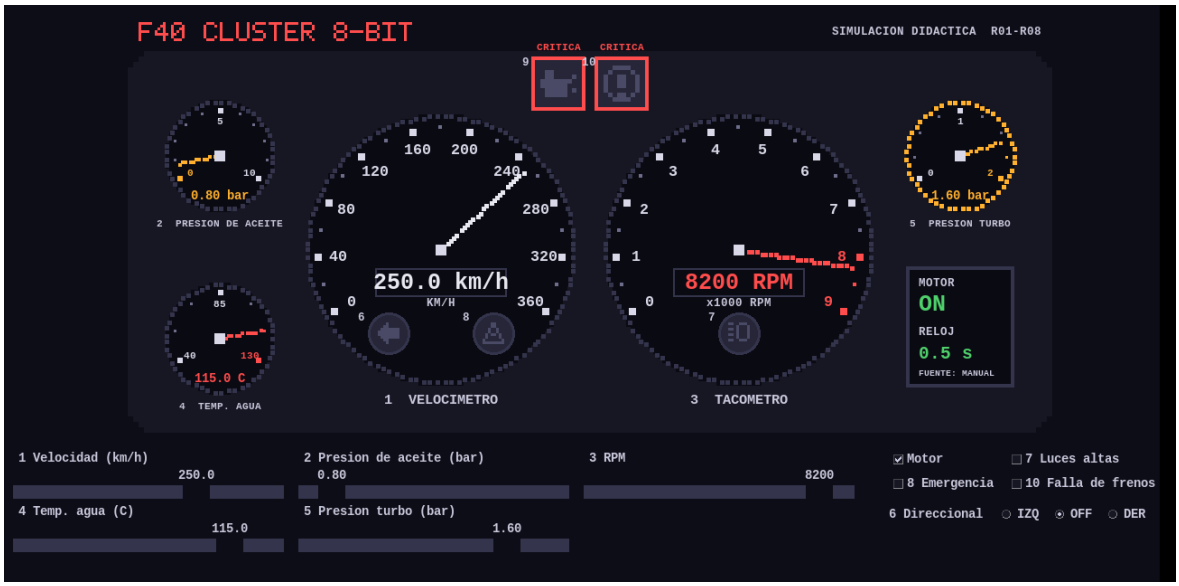


Figura 1. Tablero con advertencias activas (captura real).

3. Descripción y ficha de los diez elementos

Categorías según la guía: 2 indicadores principales (1 y 4), 3 de información en línea (2, 3 y 5), 3 testigos informativos (6, 7 y 8) y 2 testigos de seguridad (9 y 10).

N.º	Elemento	Categoría	Clave en datos/vehiculo.conf	Tipo	Unidad, rango e inicial
1	Velocímetro	Indicador principal	velocidad	Decimal (float IEEE-754 de 64 bits)	km/h; 0 a 360; inicial 0.0
2	Presión de aceite (medidor)	Información en línea	presion_aceite	Decimal	bar; 0 a 10; inicial 0.0
3	Tacómetro	Información en línea	rpm	Entero sin fracción	rpm; 0 a 9000; inicial 0
4	Temperatura de agua	Indicador principal	temp_agua	Decimal	°C; 40 a 130; inicial 40.0
5	Presión de turbo	Información en línea	presion_turbo	Decimal	bar; 0 a 2; inicial 0.0

N.º	Elemento	Categoría	Clave en datos/vehiculo.conf	Tipo	Unidad, rango e inicial
6	Direccionales	Testigo informativo	direccional	Enumeración: off, izquierda, derecha	Sin unidad; inicial off
7	Luces altas	Testigo informativo	luces_altas	Booleano codificado 0/1	Inicial 0
8	Intermitentes (emergencia)	Testigo informativo	emergencia	Booleano 0/1	Inicial 0
9	Baja presión de aceite (testigo)	Testigo de seguridad	calculado a partir de presion_aceite	Booleano calculado	Inicial apagado
10	Falla de frenos	Testigo de seguridad	falla_frenos	Booleano 0/1	Inicial 0

Ficha del elemento 1: Velocímetro

Categoría	Indicador principal
Fuente del dato	datos/vehiculo.conf, clave velocidad
Tipo de dato	Decimal (float IEEE-754 de 64 bits)
Unidad, rango y valor inicial	km/h; 0 a 360; inicial 0.0
Frecuencia de actualización	100 ms
Regla de visualización	Aguja de bloques y cifra (120.5 km/h)
Advertencia y prioridad	Sin umbral propio; R01: vuelve a 0 al apagar el motor. Prioridad normal.
Valor ausente, corrupto o fuera de rango	Ausente, texto o fuera de rango: se conserva el último valor válido y se registra ERROR (dato_invalido, fuera_de_rango o clave_faltante) en logs/errores.log.
Evento y archivo de destino	INFO valor_actualizado en logs/eventos.log
Justificación	La especificación usa décimas (120.5 km/h); un decimal evita truncar y la precisión de 64 bits sobra para una décima.

Ficha del elemento 2: Presión de aceite (medidor)

Categoría	Información en línea
Fuente del dato	datos/vehiculo.conf, clave presion_aceite
Tipo de dato	Decimal
Unidad, rango y valor inicial	bar; 0 a 10; inicial 0.0
Frecuencia de actualización	100 ms
Regla de visualización	Aguja; ámbar inmediato bajo 1 bar con motor activo
Advertencia y prioridad	Ámbar < 1 bar. Alimenta al testigo 9. Prioridad media.
Valor ausente, corrupto o fuera de rango	Igual que el elemento 1.
Evento y archivo de destino	INFO valor_actualizado
Justificación	Los umbrales 0.99 / 1.0 / 1.2 bar requieren centésimas.

Ficha del elemento 3: Tacómetro

Categoría	Información en línea
Fuente del dato	datos/vehiculo.conf, clave rpm
Tipo de dato	Entero sin fracción
Unidad, rango y valor inicial	rpm; 0 a 9000; inicial 0
Frecuencia de actualización	100 ms
Regla de visualización	Aguja; rojo ≥ 8000, normal < 7800 (histéresis)
Advertencia y prioridad	WARN tacometro_rojo (R03).
Valor ausente, corrupto o fuera de rango	Un decimal (4500.5) o texto se rechaza: no se redondea en silencio.
Evento y archivo de destino	INFO valor_actualizado; WARN tacometro_rojo
Justificación	No existen fracciones de rpm; el rango cabe en 16 bits sin

	signo (0-65535).

Ficha del elemento 4: Temperatura de agua

Categoría	Indicador principal
Fuente del dato	datos/vehiculo.conf, clave temp_agua
Tipo de dato	Decimal
Unidad, rango y valor inicial	°C; 40 a 130; inicial 40.0
Frecuencia de actualización	500 ms
Regla de visualización	Aguja; rojo > 110, normal ≤ 105
Advertencia y prioridad	WARN temperatura_rojo (R04).
Valor ausente, corrupto o fuera de rango	Igual que el elemento 1.
Evento y archivo de destino	INFO valor_actualizado; WARN temperatura_rojo
Justificación	El umbral 110.1 °C requiere décimas; la temperatura cambia lento, por eso su ciclo es de 500 ms.

Ficha del elemento 5: Presión de turbo

Categoría	Información en línea
Fuente del dato	datos/vehiculo.conf, clave presion_turbo
Tipo de dato	Decimal
Unidad, rango y valor inicial	bar; 0 a 2; inicial 0.0
Frecuencia de actualización	100 ms
Regla de visualización	Aguja; contorno ámbar > 1.5, normal ≤ 1.4
Advertencia y prioridad	WARN turbo_ambar (R05).
Valor ausente, corrupto o fuera de rango	Igual que el elemento 1.
Evento y archivo de destino	INFO valor_actualizado; WARN turbo_ambar
Justificación	Los umbrales 1.5 / 1.51 bar requieren centésimas.

Ficha del elemento 6: Direccionales

Categoría	Testigo informativo
Fuente del dato	datos/vehiculo.conf, clave direccional
Tipo de dato	Enumeración: off, izquierda, derecha
Unidad, rango y valor inicial	Sin unidad; inicial off
Frecuencia de actualización	Parpadeo de 500 ms
Regla de visualización	Testigo verde con flecha a la izquierda o a la derecha
Advertencia y prioridad	Informativo. Cede ante la emergencia (R07).
Valor ausente, corrupto o fuera de rango	Otro texto: se conserva el último valor y se registra ERROR dato_invalido.
Evento y archivo de destino	INFO valor_actualizado direccional=izquierda
Justificación	Una enumeración cerrada impide estados imposibles (izquierda y derecha a la vez).

Ficha del elemento 7: Luces altas

Categoría	Testigo informativo
Fuente del dato	datos/vehiculo.conf, clave luces_altas
Tipo de dato	Booleano codificado 0/1
Unidad, rango y valor inicial	Inicial 0
Frecuencia de actualización	Sin parpadeo
Regla de visualización	Testigo azul fijo
Advertencia y prioridad	Informativo.
Valor ausente, corrupto o fuera de rango	Cualquier valor distinto de 0 o 1 se rechaza.
Evento y archivo de destino	INFO valor_actualizado
Justificación	Dos estados; 0/1 es la codificación del ejemplo de la guía.

Ficha del elemento 8: Intermitentes (emergencia)

Categoría	Testigo informativo
Fuente del dato	datos/vehiculo.conf, clave emergencia
Tipo de dato	Booleano 0/1
Unidad, rango y valor inicial	Inicial 0
Frecuencia de actualización	Parpadeo de 500 ms
Regla de visualización	Testigo ámbar; tapa al direccional
Advertencia y prioridad	Informativo con precedencia visual (R07).
Valor ausente, corrupto o fuera de rango	Igual que el elemento 7.
Evento y archivo de destino	INFO emergencia_activada / emergencia_desactivada
Justificación	Dos estados.

Ficha del elemento 9: Baja presión de aceite (testigo)

Categoría	Testigo de seguridad
Fuente del dato	Calculado por EstadoCluster a partir de la clave presion_aceite de datos/vehiculo.conf
Tipo de dato	Booleano calculado
Unidad, rango y valor inicial	Inicial apagado
Frecuencia de actualización	Retardo de 1 s con histéresis
Regla de visualización	Rectángulo rojo con borde grueso y etiqueta CRITICA
Advertencia y prioridad	CRÍTICO. WARN prioritario testigo_9_encendido (R02).
Valor ausente, corrupto o fuera de rango	Si la presión llega inválida se evalúa con el último valor válido.
Evento y archivo de destino	WARN testigo_9_encendido; INFO testigo_9_apagado
Justificación	Se mide con marcas de tiempo, no contando ciclos: un retraso de la interfaz no altera el segundo de espera.

Ficha del elemento 10: Falla de frenos

Categoría	Testigo de seguridad
Fuente del dato	datos/vehiculo.conf, clave falla_frenos
Tipo de dato	Booleano 0/1
Unidad, rango y valor inicial	Inicial 0
Frecuencia de actualización	Parpadeo de 300 ms
Regla de visualización	Rectángulo rojo parpadeante, también con motor apagado (D1)
Advertencia y prioridad	CRÍTICO. WARN prioritario testigo_10_encendido (R08).
Valor ausente, corrupto o fuera de rango	Igual que el elemento 7.
Evento y archivo de destino	WARN testigo_10_encendido; INFO testigo_10_apagado
Justificación	Dos estados; el parpadeo más rápido lo distingue de los informativos.

Reglas de funcionamiento

Regla	Descripción
R01	encendido = 1: actualizar velocidad, RPM, temperatura, presión de aceite y turbo según su ciclo (100/500 ms). encendido = 0: velocidad y RPM a 0; las demás lecturas en un estado definido.
R02	motor_activo = 1 y p_aceite < 1.0 bar durante ≥ 1 s: encender el testigo 9 en rojo. p_aceite ≥ 1.2 bar durante ≥ 1 s: apagarlo (histéresis).
R03	rpm ≥ 8000: indicador 3 en rojo. rpm < 7800: color normal.
R04	temp_agua > 110 °C: indicador 4 en rojo. ≤ 105 °C: normal.
R05	p_turbo > 1.5 bar: contorno del indicador 5 en ámbar. ≤ 1.4 bar: sin advertencia.
R06	Direccional IZQ o DER: la lámpara 6 alterna cada 500 ms.

Regla	Descripción
	OFF: apagada.
R07	luces_altas = 1: testigo 7 azul fijo. emergencia = 1: el testigo 8 parpadea cada 500 ms y tiene precedencia visual sobre el testigo de giro.
R08	falla_frenos = 1: testigo 10 rojo parpadeante cada 300 ms hasta falla_frenos = 0.

4. Arquitectura y estructura de archivos

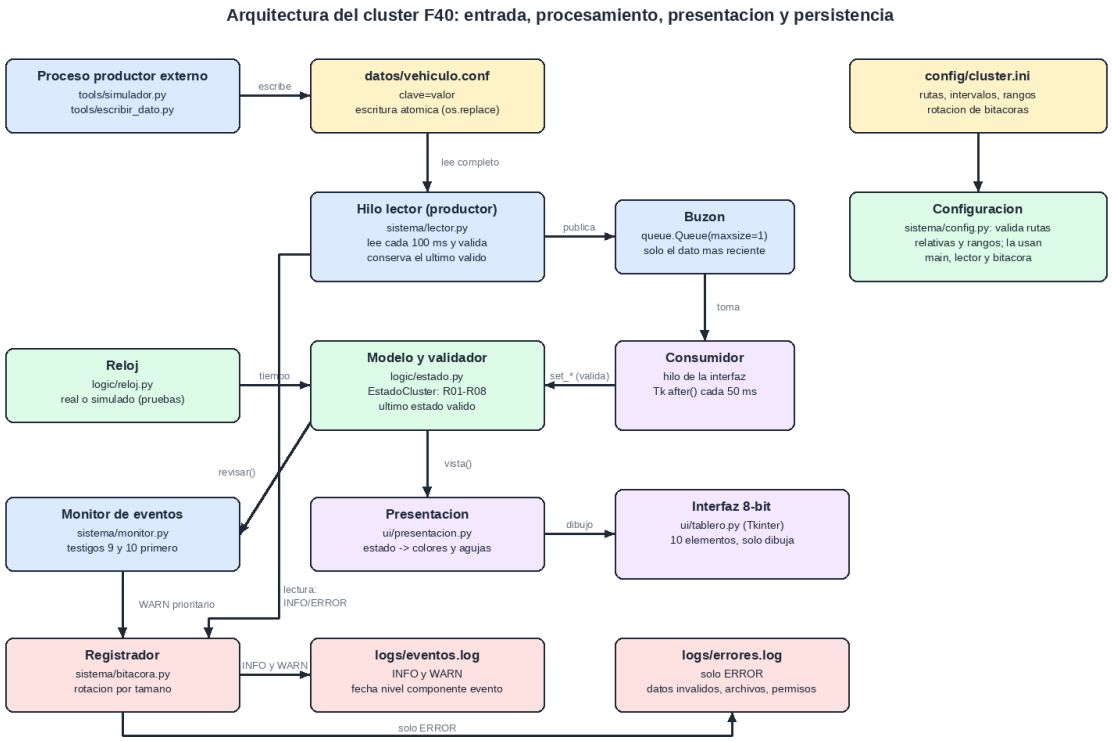


Figura 2. Componentes y flujo de datos.

Componente	Archivo	Responsabilidad	No hace
Entrada de datos	sistema/lector.py	Leer y convertir datos/vehiculo.conf en un hilo propio	Dibujar la interfaz
Validador	sistema/lector.py y logic/estado.py	Comprobar tipo, formato, rango y largo (dos capas)	Aceptar datos inválidos en silencio
Modelo del vehículo	logic/estado.py	Último estado válido de los diez elementos y reglas R01–R08	Depender del dibujo
Interfaz	ui/presentacion.py, ui/tablero.py	Representar indicadores y testigos	Modificar archivos de configuración
Registrador	sistema/bitacora.py, sistema/monitor.py	Eventos, advertencias y errores con fecha y nivel	Mezclar niveles en un archivo
Configuración	sistema/config.py, config/cluster.ini	Rutas, intervalos, rangos y rotación	Obligar a recompilar

```
cluster-f40/
├── main.py, iniciar.sh, abrir_cluster.bat   # arranque
├── logic/       # modelo (estado.py) y reloj
├── sistema/     # config, lector, bitácora, monitor
```

```
├── ui/          # presentación y tablero Tkinter
├── config/      # cluster.ini
├── datos/       # vehiculo.conf (entrada modificable)
├── logs/        # eventos.log, errores.log (se generan)
├── tests/       # pruebas y datos de prueba
├── tools/       # simulador, escribir_dato, métricas, capturas
├── aceptacion/  # ejecutor de escenarios
├── docs/        # evidencias, diagramas, matriz
├── informe/     # generadores e informes PDF
└── estudio/     # material de estudio
```

5. Representación y flujo de los datos

Tipos (bloque 4.1). Python representa los enteros con precisión arbitraria y los decimales como IEEE-754 de 64 bits; los rangos se validan explícitamente, así que no hay desbordamiento. Las RPM son enteras y un decimal se rechaza en lugar de redondearse. Caso de codificación: en Python bool es subclase de int (True == 1); por eso la validación revisa primero si el valor es booleano, y en el archivo los booleanos se codifican como 0/1 y el direccional como una enumeración de tres palabras.

- Formatos aceptados: entero [+-]dígitos; decimal con hasta 4 decimales; se rechazan nan, inf, notación científica y valores de más de 32 caracteres.
- Sin desplazamiento ni escala: el valor del archivo es el valor físico en su unidad.
- La conversión es reversible: formatear y convertir son inversas (CP-109).

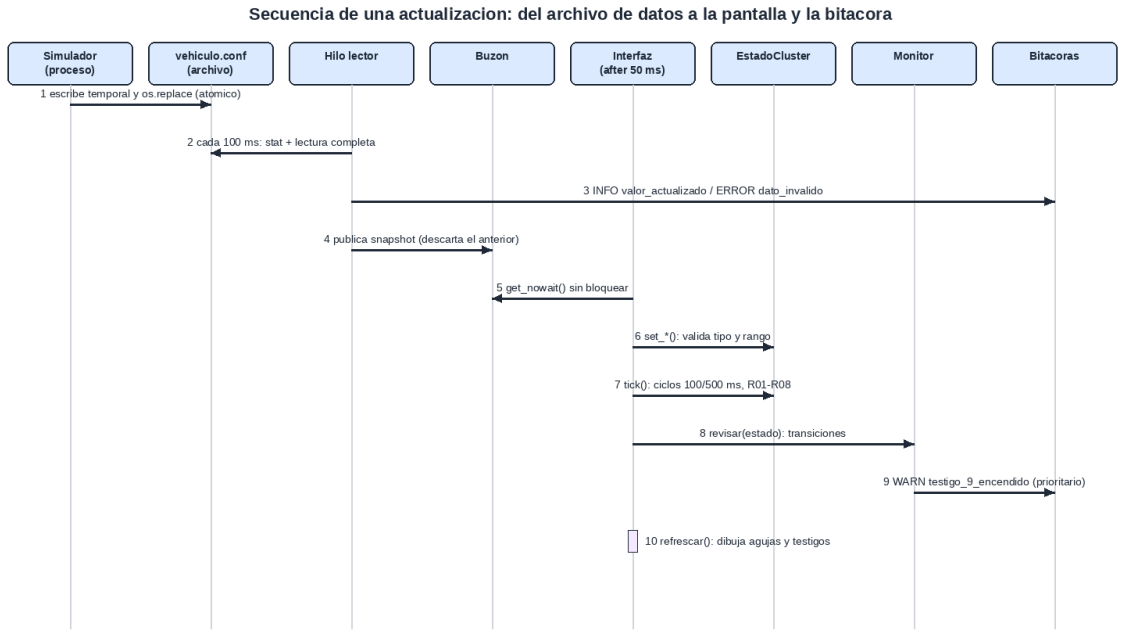


Figura 3. Secuencia de una actualización.

Archivo de datos de la demostración (datos/vehiculo.conf):

```
# Datos de entrada del cluster F40 (clave=valor). El tablero relee este archivo
# cada 100 ms: cambia un valor y guarda, sin reiniciar la aplicacion.
# Para escribir sin riesgo de lecturas parciales usa: python3 tools/escribir_dato.py
clave=valor
motor=0
```

```
velocidad=0
rpm=0
presion_aceite=0
temp_agua=40
presion_turbo=0
direccional=off
luces_altas=0
emergencia=0
falla_frenos=0
```

6. Procesos, actualización, comunicación y sincronización

Actividad	Dónde corre	Frecuencia	Prioridad
Simulador o editor (productor externo)	Proceso aparte	Cuando cambian los datos	Baja
Lectura y validación	Hilo lector (daemon), Event.wait	100 ms	Media
Consumo, tick, monitor y dibujo	Hilo principal, Tk after()	50 ms	Alta: el monitor de seguridad va primero
Escritura de bitácoras	Quien detecta el evento; logging con candados	Por evento	WARN de seguridad inmediato

La tarea más crítica es la evaluación de los testigos 9 y 10. Si el ciclo se retrasa, la pantalla se actualiza tarde, pero las reglas no se equivocan: el retardo de 1 s del testigo 9 se mide con marcas de tiempo y los parpadeos con el reloj absoluto. No hay espera activa: el hilo lector duerme con Event.wait y la interfaz con after().

Comunicación: productor externo → archivo (escritura atómica) → hilo lector → buzón queue.Queue(maxsize=1) → interfaz. Ningún objeto se modifica desde dos hilos. La prueba P08 (CP-98) ejecuta 300 escrituras mientras el hilo lee cada 1 ms y comprueba que cada dato leído es consistente.

¿Por qué ese formato de entrada?

Necesidad	Que los diez valores se cambien desde fuera, sin tocar el código.
Alternativas	JSON; CSV; archivo clave=valor.
Criterios	Sencillez para editar a mano, validación línea por línea, seguridad.
Decisión	Archivo de texto clave=valor (datos/vehiculo.conf), el formato del ejemplo de la guía.
Consecuencia	Se edita con cualquier editor y cada línea se valida por separado; no admite estructuras anidadas (no las necesita).
Evidencia	sistema/lector.py: parsear, convertir; CP-91 a CP-96, CP-107.

¿Por qué esa frecuencia de actualización?

Necesidad	Mostrar cada cambio sin desperdiciar CPU.
Alternativas	Leer y dibujar en cada evento; ciclos de 10 ms; ciclos de 100/50 ms.
Criterios	Respuesta, consumo, prioridad.
Decisión	Lectura cada 100 ms (el ciclo más rápido de la especificación) y refresco cada 50 ms (la mitad, para no perder cambios).

Consecuencia	Respuesta de 100 ms o menos con bajo consumo; los valores ya leídos se ignoran si el archivo no cambió (fecha y tamaño).
Evidencia	config/cluster.ini; intervalo real medido en docs/evidencias/metricas.txt.

¿Qué mecanismo comunica los componentes y qué alternativa se descartó?

Necesidad	Pasar los datos del lector a la interfaz sin bloquearla.
Alternativas	Variables globales compartidas; tubería; cola en memoria.
Criterios	Seguridad entre hilos, sencillez, memoria acotada.
Decisión	Entre procesos: archivo con escritura atómica. Entre hilos: queue.Queue(maxsize=1), un buzón productor-consumidor.
Consecuencia	Las variables globales se descartaron por condiciones de carrera; la cola ya trae sus candados. Solo se conserva el dato más reciente.
Evidencia	sistema/lector.py: HiloLector, publicar, Consumidor; CP-98, CP-108.

¿Cómo se evita que dos tareas modifiquen el mismo recurso a la vez?

Necesidad	El lector y la interfaz corren en hilos distintos.
Alternativas	Candados en EstadoCluster; un solo dueño por recurso.
Criterios	Riesgo de interbloqueo, sencillez.
Decisión	Un solo dueño: solo el hilo de la interfaz modifica EstadoCluster y la ventana (Tkinter no es seguro entre hilos). El lector solo produce copias.
Consecuencia	Sin candados en el modelo y sin interbloqueos. La bitácora usa los candados internos de logging y el contador usa un Lock.
Evidencia	main.py (Aplicacion); sistema/bitacora.py; CP-98, CP-115.

¿Qué datos se conservan en memoria y cuáles se persisten?

Necesidad	Estado actual rápido, pero historial y errores recuperables.
Alternativas	Guardar todo en disco; todo en memoria; separación.
Criterios	Rendimiento, durabilidad, recuperación.
Decisión	En memoria: el estado de los diez elementos y el último valor válido. En disco: configuración, datos de entrada, eventos y errores, en archivos separados.
Consecuencia	Memoria constante (sin historial en RAM); la bitácora es la historia y rota por tamaño.
Evidencia	logic/estado.py; sistema/lector.py (ultimo_valido); logs/; CP-104, metricas.txt.

¿Por qué los permisos y controles son suficientes?

Necesidad	Evitar que otro usuario manipule los datos o lea las bitácoras.
Alternativas	Confiar en los permisos por defecto; restringir.
Criterios	Mínimo privilegio, detección.
Decisión	Bitácoras 640, carpeta de logs 750, archivo de datos escrito con 640; un archivo de datos escribible por cualquiera se rechaza.

Consecuencia	Otro usuario no puede inyectar datos ni leer las bitácoras; el dueño conserva el control.
Evidencia	sistema/bitacora.py, sistema/lector.py; CP-101 a CP-103.

¿Cómo se recupera el sistema después de un dato corrupto o una interrupción?

Necesidad	Que un error no detenga el tablero.
Alternativas	Detener y reiniciar; ignorar el error; conservar el último válido.
Criterios	Disponibilidad, trazabilidad.
Decisión	Cada clave conserva su último valor válido; el error se registra una vez; al corregir el archivo se aplica sin reiniciar.
Consecuencia	El tablero sigue mostrando un estado coherente; durante el error el valor puede estar desactualizado (limitación aceptada).
Evidencia	CP-95, CP-97, CP-100, CP-110, CP-111; docs/evidencias/bitacoras/con_errores.

7. Memoria, persistencia y almacenamiento

- En memoria: un objeto EstadoCluster de tamaño fijo, el diccionario del último valor válido (10 claves) y un buzón de capacidad 1. No hay historial en RAM, así que la memoria no crece con el tiempo.
- La ventana crea sus figuras de pixeles una sola vez y después solo las mueve o recolorea; guarda el último valor dibujado para no repetir trabajo (localidad: se reutilizan los mismos objetos).
- Persistencia: config/cluster.ini y datos/vehiculo.conf se sobrescriben completos (acceso secuencial); las bitácoras solo se agregan al final y rotan por tamaño (max_bytes y número de respaldos).
- Durabilidad: la escritura atómica hace fsync antes del renombrado; tras una interrupción queda la versión vieja completa o la nueva completa.

Métricas reales de la ejecución prolongada (docs/evidencias/metricas.txt):

```
Prueba P12 - ejecucion prolongada con metricas reales
Duracion medida: 60.0 s (solicitada 60 s)
Configuracion: lectura cada 100 ms, refresco cada 50 ms, bitacoras de 16384 bytes con 2
respaldos
Productor: proceso tools/simulador.py escribiendo el archivo de datos cada 100 ms (escritura
atomica)

Intervalo real de actualizacion del tablero (ms):
  ciclos=1169 promedio=51.3 mediana=51.0 p95=52.7 maximo=146.1

Datos procesados:
  lecturas completas del archivo=391 snapshots aplicados por la interfaz=391

  eventos registrados: INFO=1179 WARN=15 ERROR=0
  lineas presentes en bitacoras: eventos=353 errores=0 (las antiguas se descartan al rotar)

Memoria (RSS):
  inicio=28780 KB fin=29100 KB diferencia=320 KB

Bitacoras (bytes):
  inicio={'errores.log': 0, 'eventos.log': 110}
  fin={'errores.log': 0, 'eventos.log': 1429, 'eventos.log.1': 16325, 'eventos.log.2': 16330}
```

```
  total final=34084 limite teorico=49152 acotado=si

ps al inicio:
PID    PPID STAT %CPU %MEM   RSS    VSZ CMD
   1198    1185 Sl    7.0  1.4 28780 120672 python tools/ejecucion_prolongada.py --duracion-s
60
ps al final:
PID    PPID STAT %CPU %MEM   RSS    VSZ CMD
   1198    1185 Sl    3.0  1.4 29100 120672 python tools/ejecucion_prolongada.py --duracion-s
60
```

8. Seguridad, amenazas y controles

Amenaza	Preventivo	Detección	Recuperación	Evidencia
Datos de entrada inválidos o manipulados	Validación de tipo, formato, rango y largo antes de usar cada valor	ERROR en errores.log con el texto recibido	Se conserva el último valor válido	CP-94, CP-95, CP-107
Otro usuario modifica el archivo de datos	Permisos 640; rechazo de archivos escribibles por cualquiera	ERROR permisos_inseguros	Al corregir el permiso se vuelve a leer sin reiniciar	CP-101, CP-102
Lectura de un archivo a medio escribir	Escritura en temporal + os.replace atómico	Validación de cada clave	Se descarta el dato incompleto	CP-98, CP-112
Llenado del disco por bitácoras	Rotación por tamaño con respaldos fijos	Tamaño medido en la prueba P12	Los archivos más viejos se descartan	CP-104, metricas.txt
Rutas fuera del proyecto en la configuración	Solo rutas relativas dentro de la carpeta del proyecto	ERROR ruta_rechazada	Se usa la ruta por defecto	CP-106
Inyección de comandos	El programa no arma comandos de sistema con datos; ps se llama con lista de argumentos	No aplica	No aplica	Revisión de código (subprocess sin shell)

No hay contraseñas, tokens ni datos sensibles en el código. Las rutas son relativas al proyecto. El programa no ejecuta comandos construidos con datos de entrada.

9. Manejo de errores y bitácoras

Cada línea tiene fecha ISO 8601 con zona horaria, nivel (INFO, WARN, ERROR), componente, evento, detalle y resultado. logs/eventos.log guarda la operación normal y las advertencias; logs/errores.log guarda solo errores. Un mismo error se registra una vez mientras dura, para no inundar la bitácora (CP-110).

Ejecución exitosa

docs/evidencias/bitacoras/exitosa/eventos.log:

```
2026-11-10T17:40:53-06:00 INFO  sistema inicio modo=archivo=vehiculo.conf
2026-11-10T17:40:53-06:00 INFO  lector_datos valor_actualizado motor=1 resultado=aceptado
2026-11-10T17:40:53-06:00 INFO  lector_datos valor_actualizado presion_aceite=5.0
resultado=aceptado
2026-11-10T17:40:53-06:00 INFO  lector_datos valor_actualizado temp_agua=85.0
resultado=aceptado
2026-11-10T17:40:53-06:00 INFO  modelo_vehiculo motor_encendido (R01)
```

```
2026-11-10T17:40:53-06:00 INFO  lector_datos valor_actualizado velocidad=72.0
resultado=aceptado
2026-11-10T17:40:53-06:00 INFO  lector_datos valor_actualizado rpm=2450 resultado=aceptado
2026-11-10T17:40:53-06:00 INFO  lector_datos valor_actualizado presion_turbo=0.8
resultado=aceptado
2026-11-10T17:40:53-06:00 INFO  lector_datos valor_actualizado velocidad=180.0
resultado=aceptado
2026-11-10T17:40:53-06:00 INFO  lector_datos valor_actualizado rpm=8100 resultado=aceptado
2026-11-10T17:40:53-06:00 INFO  lector_datos valor_actualizado presion_turbo=1.6
resultado=aceptado
2026-11-10T17:40:53-06:00 INFO  lector_datos valor_actualizado luces_altas=1 resultado=aceptado
2026-11-10T17:40:53-06:00 WARN  modelo_vehiculo tacometro_rojo rpm=8100 (R03)
2026-11-10T17:40:53-06:00 WARN  modelo_vehiculo turbo_ambar presion_turbo=1.6 bar (R05)
2026-11-10T17:40:53-06:00 INFO  lector_datos valor_actualizado rpm=6000 resultado=aceptado
2026-11-10T17:40:53-06:00 INFO  lector_datos valor_actualizado presion_turbo=0.9
resultado=aceptado
2026-11-10T17:40:53-06:00 INFO  lector_datos valor_actualizado direccional=izquierda
resultado=aceptado
2026-11-10T17:40:53-06:00 INFO  modelo_vehiculo tacometro_normal rpm=6000 (R03)
```

docs/evidencias/bitacoras/exitosa/errores.log:

```
(vacío: no hubo errores)
```

Ejecución con errores

docs/evidencias/bitacoras/con_errores/eventos.log:

```
2026-11-10T17:40:53-06:00 INFO  sistema inicio modo=archivo=vehiculo.conf
2026-11-10T17:40:53-06:00 INFO  lector_datos archivo_recuperado ruta=vehiculo.conf
resultado=aceptado
2026-11-10T17:40:53-06:00 INFO  lector_datos valor_actualizado motor=1 resultado=aceptado
2026-11-10T17:40:53-06:00 INFO  lector_datos valor_actualizado presion_aceite=5.0
resultado=aceptado
2026-11-10T17:40:53-06:00 INFO  modelo_vehiculo motor_encendido (R01)
2026-11-10T17:40:53-06:00 INFO  lector_datos archivo_recuperado ruta=vehiculo.conf
resultado=aceptado
2026-11-10T17:40:53-06:00 INFO  lector_datos valor_actualizado presion_aceite=0.4
resultado=aceptado
2026-11-10T17:40:53-06:00 WARN  monitor_seguridad testigo_9_encendido presion_aceite=0.4 bar
durante >= 1 s (R02)
2026-11-10T17:40:53-06:00 INFO  lector_datos valor_actualizado presion_aceite=5.0
resultado=aceptado
2026-11-10T17:40:53-06:00 INFO  monitor_seguridad testigo_9_apagado presion_aceite=5 bar
2026-11-10T17:40:53-06:00 INFO  sistema fin info=9 warn=1 error=5
```

docs/evidencias/bitacoras/con_errores/errores.log:

```
2026-11-10T17:40:53-06:00 ERROR lector_datos archivo_inexistente ruta=vehiculo.conf se conserva
el ultimo valor valido resultado=rechazado
2026-11-10T17:40:53-06:00 ERROR lector_datos dato_invalido temp_agua entrada="abc" (se espera
un numero decimal) resultado=rechazado
2026-11-10T17:40:53-06:00 ERROR lector_datos fuera_de_rango velocidad=500 (rango 0-360)
resultado=rechazado
2026-11-10T17:40:53-06:00 ERROR lector_datos clave_faltante rpm se conserva 0
resultado=rechazado
```

```
2026-11-10T17:40:53-06:00 ERROR lector_datos permisos_inseguros ruta=vehiculo.conf modo=0o666
escribible por cualquiera resultado=rechazado
```

Configuración vigente (config/cluster.ini):

```
[rutas]
datos = datos/vehiculo.conf
logs = logs
[intervalos]
lectura_ms = 100
refresco_ms = 50
[bitacora]
max_bytes = 262144
respaldos = 3
[rangos]
velocidad = 0, 360
rpm = 0, 9000
presion_aceite = 0, 10
temp_agua = 40, 130
presion_turbo = 0, 2
```

10. Pruebas, resultados, limitaciones y conclusiones

ID	Caso	Entrada	Procedimiento	Esperado	Obtenido	Estado
P01	Valor normal	velocidad=72	Escribir el archivo, leer y aplicar un ciclo	Indicador actualizado y evento INFO	1 ejecuciones de CP-91: 1 aprobadas, 0 fallidas, 0 omitidas.	Aprobada
P02	Límite inferior	0 km/h, 0 rpm, 0 bar, 40 °C, 0 bar	Leer el archivo con todos los mínimos	Valores aceptados sin error	1 ejecuciones de CP-92: 1 aprobadas, 0 fallidas, 0 omitidas.	Aprobada
P03	Límite superior	360 km/h, 9000 rpm, 10 bar, 130 °C, 2 bar	Leer el archivo con todos los máximos	Valores aceptados sin error	1 ejecuciones de CP-93: 1 aprobadas, 0 fallidas, 0 omitidas.	Aprobada
P04	Fuera de rango	361, -1, 9001, 39.9, 2.01	Cambiar un valor fuera de rango	Rechazo controlado, aviso y ERROR	6 ejecuciones de CP-94, CP-77: 6 aprobadas, 0 fallidas, 0 omitidas.	Aprobada
P05	Formato incorrecto	abc, 4500.5, si, arriba, nan	Escribir texto donde va un número	Continúa con el último valor válido	5 ejecuciones de CP-95: 5 aprobadas, 0 fallidas, 0 omitidas.	Aprobada
P06	Dato ausente	Archivo vacío; clave faltante	Leer sin claves	Estado definido y error registrado	1 ejecuciones de CP-96: 1 aprobadas, 0 fallidas, 0 omitidas.	Aprobada
P07	Archivo inexistente	Ruta retirada	Leer sin archivo y luego	No termina; se recupera	1 ejecuciones de CP-97: 1 aprobadas, 0	Aprobada

ID	Caso	Entrada	Procedimiento	Esperado	Obtenido	Estado
			restaurarlo		fallidas, 0 omitidas.	
P08	Actualización concurrente	300 escrituras atómicas mientras el hilo lee cada 1 ms	Comprobar cada dato leído	Sin valores parciales ni corrupción	1 ejecuciones de CP-98: 1 aprobadas, 0 fallidas, 0 omitidas.	Aprobada
P09	Advertencia	presion_aceite =0.5 sostenida; falla_frenos=1	Avanzar 1.2 s	Testigo visible y WARN prioritario	1 ejecuciones de CP-99: 1 aprobadas, 0 fallidas, 0 omitidas.	Aprobada
P10	Recuperación	rpm=abc y luego rpm=2500	Corregir la entrada	Retoma sin reinicio	2 ejecuciones de CP-100, CP-77: 2 aprobadas, 0 fallidas, 0 omitidas.	Aprobada
P11	Permisos	Archivo 000, archivo 666; bitácoras	Leer y comprobar modos	Operación rechazada y registrada; bitácoras 640	3 ejecuciones de CP-101, CP-102, CP-103: 3 aprobadas, 0 fallidas, 0 omitidas.	Aprobada
P12	Ejecución prolongada	3000 ciclos acelerados y ejecución real con simulador	Medir bitácoras, memoria e intervalo	Sin crecimiento ilimitado	1 ejecuciones de CP-104: 1 aprobadas, 0 fallidas, 0 omitidas.	Aprobada

Indicador	Valor
Pruebas obligatorias P01–P12	12/12 aprobadas
Ejecuciones automatizadas (pytest)	145: 145 aprobadas, 0 fallidas, 0 omitidas
Cobertura total (líneas y ramas)	99.2%
Plataforma	Ubuntu 22.04.5 LTS (kernel 5.15.0-190-generic); Python 3.10.12

Limitaciones

- Durante un error de entrada el tablero muestra el último valor válido, que puede estar desactualizado.
- La interfaz se probó con Tkinter real sobre una pantalla virtual (Xvfb) y en Windows; no en otras plataformas.
- El editor de texto que guarde sobre el mismo archivo puede producir una lectura incompleta; la validación la rechaza y se usa la siguiente. Para cambios en vivo se recomienda tools/escribir_dato.py (atómico).
- Los permisos POSIX no aplican en Windows; ahí esas pruebas se omiten.
- Las categorías de los elementos 2, 3 y 5 (información en línea) deben confirmarse con el profesor.

Conclusiones

Las 12 de 12 pruebas obligatorias están aprobadas y las 145 ejecuciones automatizadas registran 0 fallos. El sistema recibe sus datos de un archivo externo, los valida en dos capas, se recupera de datos corruptos o ausentes sin reiniciar, separa eventos y errores y mantiene acotados la memoria y el disco.

Anexo: colaboración de inteligencia artificial

Claude Code (Anthropic) colaboró con el Equipo 3: escribió el código, las pruebas, los scripts de evidencia y este informe. El equipo tomó las decisiones de interpretación, ejecutó cada etapa y validó los resultados. El detalle está en docs/bitacora_ia.md y en el informe de pruebas.